

Инструкция и образец

ПП.04.01 «Сопровождение и обслуживание программного обеспечения компьютерных систем»

Сборка приложения C# Windows Forms, затем подключение CSV · 30 индивидуальных вариантов

09.02.07 Информационные системы и программирование · программист

ПМ.04 Сопровождение и обслуживание программного обеспечения компьютерных систем

Учебно-методический комплект

Сервисный журнал учебного центра

Отдельный пример: проектор и ноутбук, одна неполная запись принтера.



Карта демонстрации: от приложения к исправлению

Материалы для работы

[Скачать исходный проект ServiceDesk \(ZIP\)](#)

Начните с него: соберите и проверьте приложение без CSV.

[Скачать заготовку подключения CSV \(ZIP\)](#)

Откройте после первой проверки, чтобы изучить дефекты импорта и экспорта.

[Скачать демонстрационный CSV с неполной строкой \(demo.csv\)](#)

Используйте для воспроизведения неполной строки и исправленного импорта.

[Скачать демонстрационный CSV без дефектной строки \(demo-valid.csv\)](#)

Используйте для отдельной проверки экспорта и повторного импорта.

[Скачать исправленный демонстрационный проект \(ZIP\)](#)

Сверьтесь после самостоятельной диагностики; это решение демонстрационной области.



Исходная система

Демонстрационная предметная область — сервисный журнал учебного центра. Объекты «Проектор» и «Ноутбук» используются только для объяснения приёмов. Стартовый ServiceDesk-source.zip не имеет CSV и позволяет отдельно проверить таблицу и счётчик. После этого используется CSV-заготовка с намеренными дефектами, затем исправленный пример для сверки.

Связь этапов

Каждый следующий этап опирается на проверенный предыдущий: работа окна → контракт данных → воспроизведение D1–D3 → правка по причине → регрессия → обновление. Если пропустить исходную проверку, сообщение импорта нельзя уверенно отличить от ошибки формы. Если не зафиксировать вход и ожидание, невозможно доказать, что изменение устранило исходный дефект.

Где лежат файлы

Прямые ссылки под этой страницей скачивают архив исходного проекта, заготовку интеграции, демонстрационный CSV и исправленный пример. Распакуйте каждый архив в отдельную папку. Данные demo.csv и demo-valid.csv положите в отдельный каталог данных, чтобы не путать их с исходниками. Все пути в инструкции относятся к содержимому скачанных архивов, а не к служебной папке преподавателя.

Открытие проекта и устройство решения

Материалы для работы

[Скачать исходный проект ServiceDesk \(ZIP\)](#)

Начните с него: соберите и проверьте приложение без CSV.

Действия в Visual Studio

Выберите «Файл → Открыть → Проект или решение» и укажите ServiceDesk.csproj из распакованного ServiceDesk-source.zip. В «Обозревателе решений» откройте Program.cs. Если компоненты разработки настольных приложений .NET отсутствуют, используйте установленную на учебном компьютере среду с поддержкой Windows Forms; установку новых компонентов согласуйте с администратором. Выберите «Сборка → Собрать решение», затем F5.

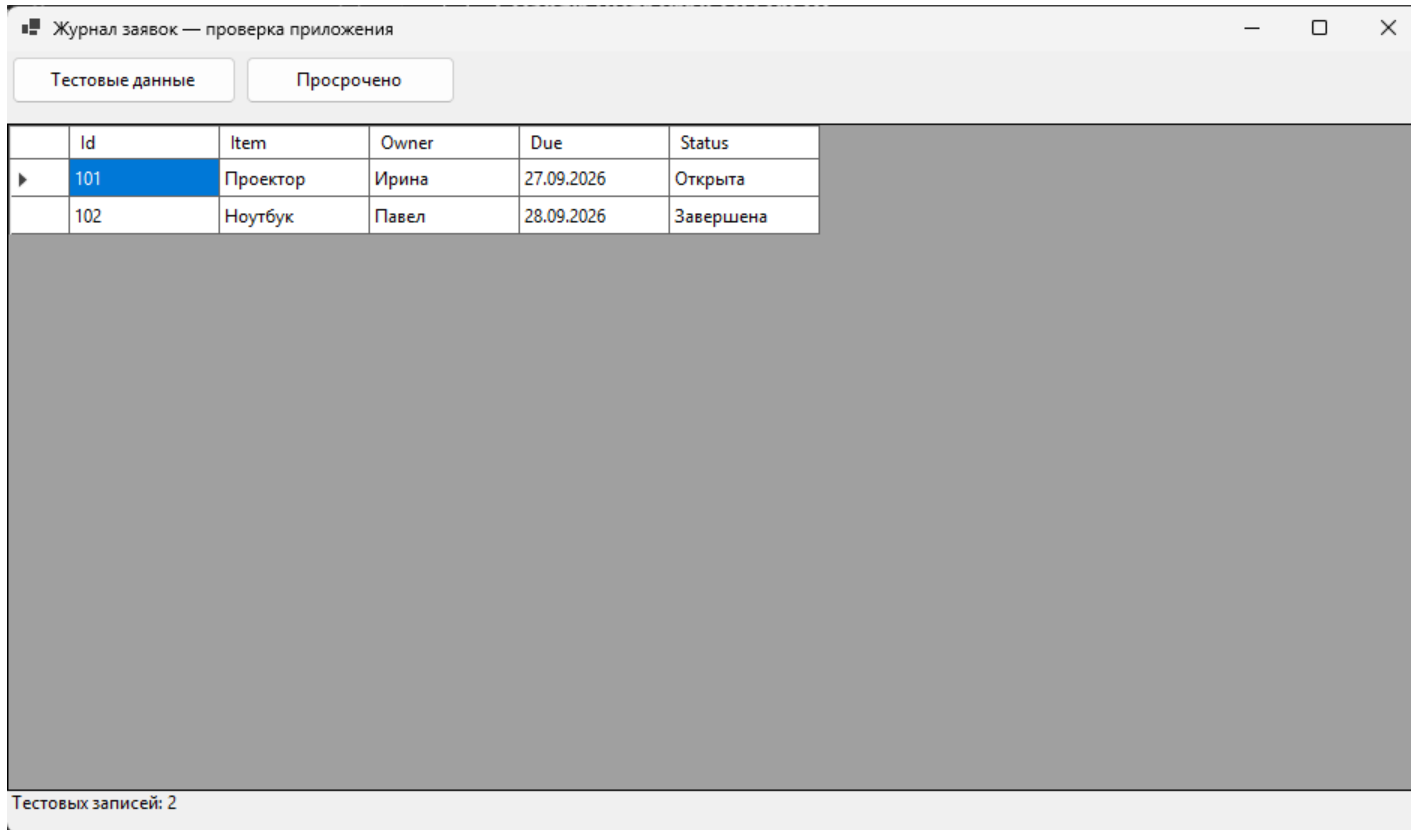
Что вы открыли

ServiceDesk.csproj задаёт Windows Forms и целевую платформу. Program.cs содержит модель Ticket, генератор TestData и форму MainForm. Ticket хранит пять значений; TestData создаёт две записи и считает просроченные. MainForm создаёт DataGridView, строку состояния и кнопки. Этот небольшой исходник удобно проверить до добавления файлового обмена.

Первый контроль

Нажмите «Тестовые данные», затем «Просрочено». Таблица показывает проектор и ноутбук, статус — сначала две записи, затем одну просроченную. Сохраните снимки и исходную копию. Если проект не собирается, откройте «Список ошибок» и исправьте проблему среды или проекта, прежде чем переходить к CSV.

Приложение до подключения CSV

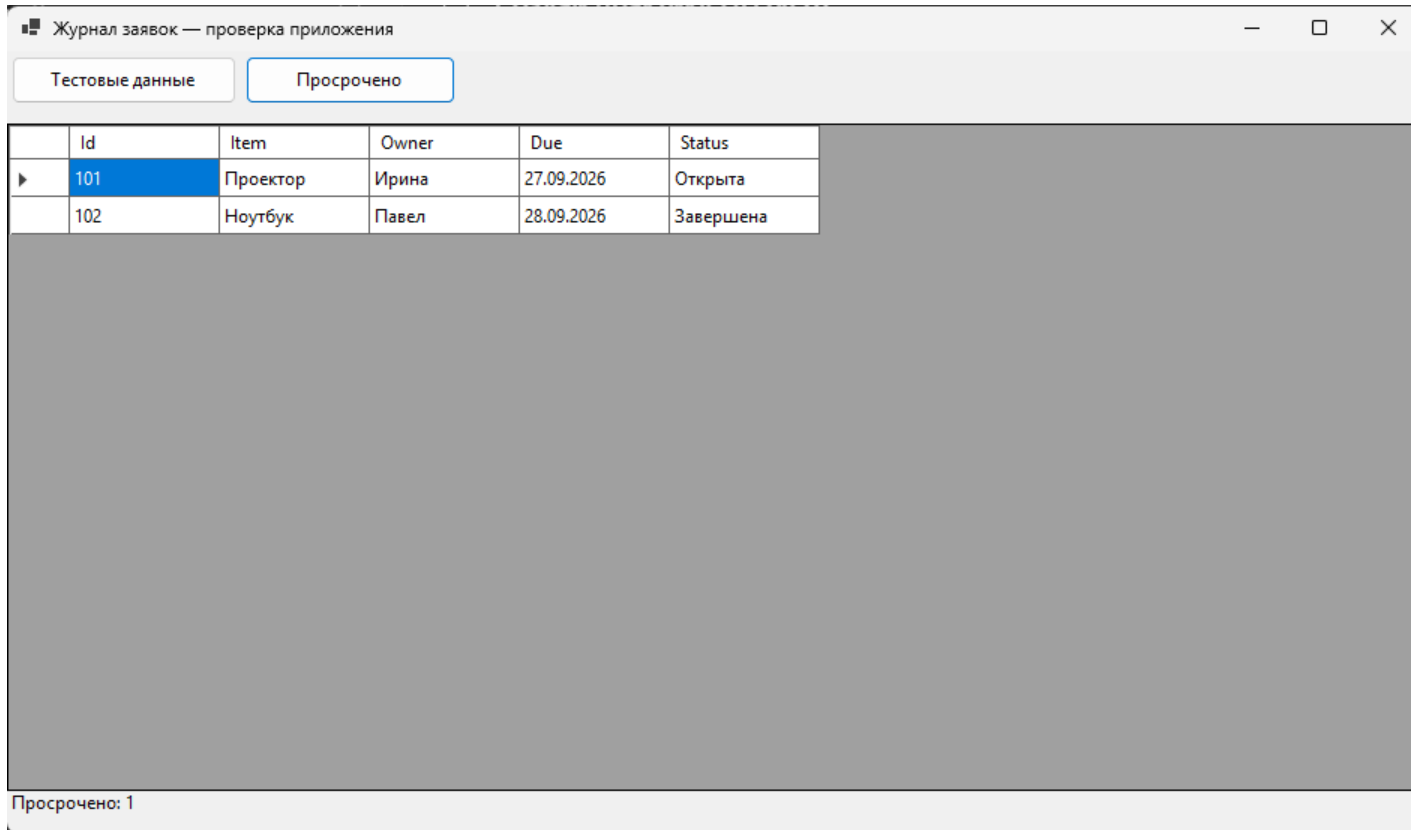


	Id	Item	Owner	Due	Status
►	101	Проектор	Ирина	27.09.2026	Открыта
	102	Ноутбук	Павел	28.09.2026	Завершена

Тестовых записей: 2

Реальная сборка стартового проекта: две встроенные записи в таблице; CSV ещё не используется.

Проверка расчёта на встроенных данных



Журнал заявок — проверка приложения

Тестовые данные Просрочено

	Id	Item	Owner	Due	Status
▶	101	Проектор	Ирина	27.09.2026	Открыта
	102	Ноутбук	Павел	28.09.2026	Завершена

Просрочено: 1

После нажатия «Просрочено» стартовое приложение показывает 1.

Модель и контрольные записи

Где выполнять действие

В ServiceDesk-source.zip откройте Program.cs. Не создавайте новый файл: следующие строки уже находятся в исходном проекте. Проследите от using к Ticket и TestData. При адаптации своего варианта позднее меняются значения Item и Owner, но состав полей модели сохраняется.

Program-starter.cs: строки 1–18

```
using System.Windows.Forms;

namespace ServiceDesk;

internal record Ticket(int Id, string Item, string Owner, DateTime Due, string Status);

internal static class TestData
{
    public static List<Ticket> Create(DateTime today) =>
    [
        new Ticket(101, "Проектор", "Ирина", today.AddDays(-2), "Открыта"),
        new Ticket(102, "Ноутбук", "Павел", today.AddDays(-1), "Завершена")
    ];

    public static int CountOverdue(IEnumerable<Ticket> tickets, DateTime today) =>
        tickets.Count(t => t.Status != "Завершена" && t.Due.Date < today.Date);
}
```

Почему это работает и как проверить

TestData.Create(DateTime.Today) создаёт одну открытую запись с прошедшей датой и одну завершённую. CountOverdue исключает завершённые записи и сравнивает только календарную дату. Повторите проверку кнопки после изменения демонстрационных значений своего варианта.

Форма, таблица и кнопки

Где выполнять действие

Продолжайте в том же Program.cs, класс MainForm. Найдите объявления grid, status и tickets. Если повторяете сборку вручную, добавьте эти поля внутрь класса до конструктора.

Program-starter.cs: строки 19–24

```
internal sealed class MainForm : Form
{
    private readonly DataGridView grid = new() { Dock = DockStyle.Fill, ReadOnly = true, AutoGenerateColumns = true };
    private readonly Label status = new() { Dock = DockStyle.Bottom, Height = 28, Text = "Сначала загрузите встроенные тестовые данные" };
    private List<Ticket> tickets = new();
}
```

Почему это работает и как проверить

Таблица хранит отображение, status — обратную связь оператору, tickets — текущие записи. Поля должны быть доступны и конструктору, и обработчикам; соберите проект после добавления.

Конструктор и реакции кнопок

Где выполнять действие

В MainForm после полей добавьте конструктор. Он задаёт размеры окна, создаёт панель и две кнопки, привязывает Click и располагает элементы. При добавлении новой кнопки позже используйте ту же панель.

Program-starter.cs: строки 25–41

```
public MainForm(bool showTestData = false)
{
    Text = "Журнал заявок – проверка приложения";
    Width = 960;
    Height = 560;
    var bar = new FlowLayoutPanel { Dock = DockStyle.Top, Height = 48 };
    var load = new Button { Text = "Тестовые данные", Width = 150, Height = 32 };
    var count = new Button { Text = "Просрочено", Width = 140, Height = 32 };
    load.Click += (_, _) => LoadTestData();
    count.Click += (_, _) => status.Text = $"Просрочено: {TestData.CountOverdue(tickets, DateTime.Today)}";
    bar.Controls.AddRange(new Control[] { load, count });
    Controls.Add(grid);
    Controls.Add(status);
    Controls.Add(bar);
    if (showTestData) Shown += (_, _) => LoadTestData();
}
```

Почему это работает и как проверить

«Тестовые данные» вызывает LoadTestData, а «Просрочено» использует текущий tickets. Проверьте, что обе кнопки видны и нажатие обновляет таблицу или статус.

Загрузка тестовых данных и запуск

Где выполнять действие

В конце MainForm найдите LoadTestData и Main. При ручном повторении добавьте методы после конструктора внутри класса. Точка входа Main запускает окно, а режим --self-test проверяет данные без GUI.

Program-starter.cs: строки 42–61

```
private void LoadTestData()
{
    tickets = TestData.Create(DateTime.Today);
    grid.DataSource = tickets.ToList();
    status.Text = $"Тестовых записей: {tickets.Count}";
}

[STAThread]
private static void Main(string[] args)
{
    if (args.Length == 1 && args[0] == "--self-test")
    {
        var data = TestData.Create(DateTime.Today);
        Console.WriteLine($"test-records={data.Count}; overdue={TestData.CountOverdue(data, DateTime.Today)}");
        return;
    }
    ApplicationConfiguration.Initialize();
    Application.Run(new MainForm(args.Length == 1 && args[0] == "--demo"));
}
```

Почему это работает и как проверить

LoadTestData привязывает копию списка к таблице и обновляет статус. Запустите F5 и повторите два действия; затем можно выполнить dotnet run --project ServiceDesk.csproj -- --self-test и получить test-records=2; overdue=1. Только после этого переходите к файлу.

Как добавить CSV и сохранить контрольную точку

Материалы для работы

[Скачать заготовку подключения CSV \(ZIP\)](#)

Откройте после первой проверки, чтобы изучить дефекты импорта и экспорта.

[Скачать демонстрационный CSV с неполной строкой \(demo.csv\)](#)

Используйте для воспроизведения неполной строки и исправленного импорта.

[Скачать демонстрационный CSV без дефектной строки \(demo-valid.csv\)](#)

Используйте для отдельной проверки экспорта и повторного импорта.

Рабочая копия

Скопируйте проверенный стартовый проект в новую папку. Рядом распакуйте ServiceDesk-csv-draft.zip; в нём уже показан первый вариант подключения CSV. Откройте оба Program.cs и сравните классы: добавились CsvStore, кнопки «Открыть CSV» и «Экспорт CSV», диалоги выбора файлов. Сохраняйте исходную копию неизменной, чтобы можно было вернуться к проверенной таблице.

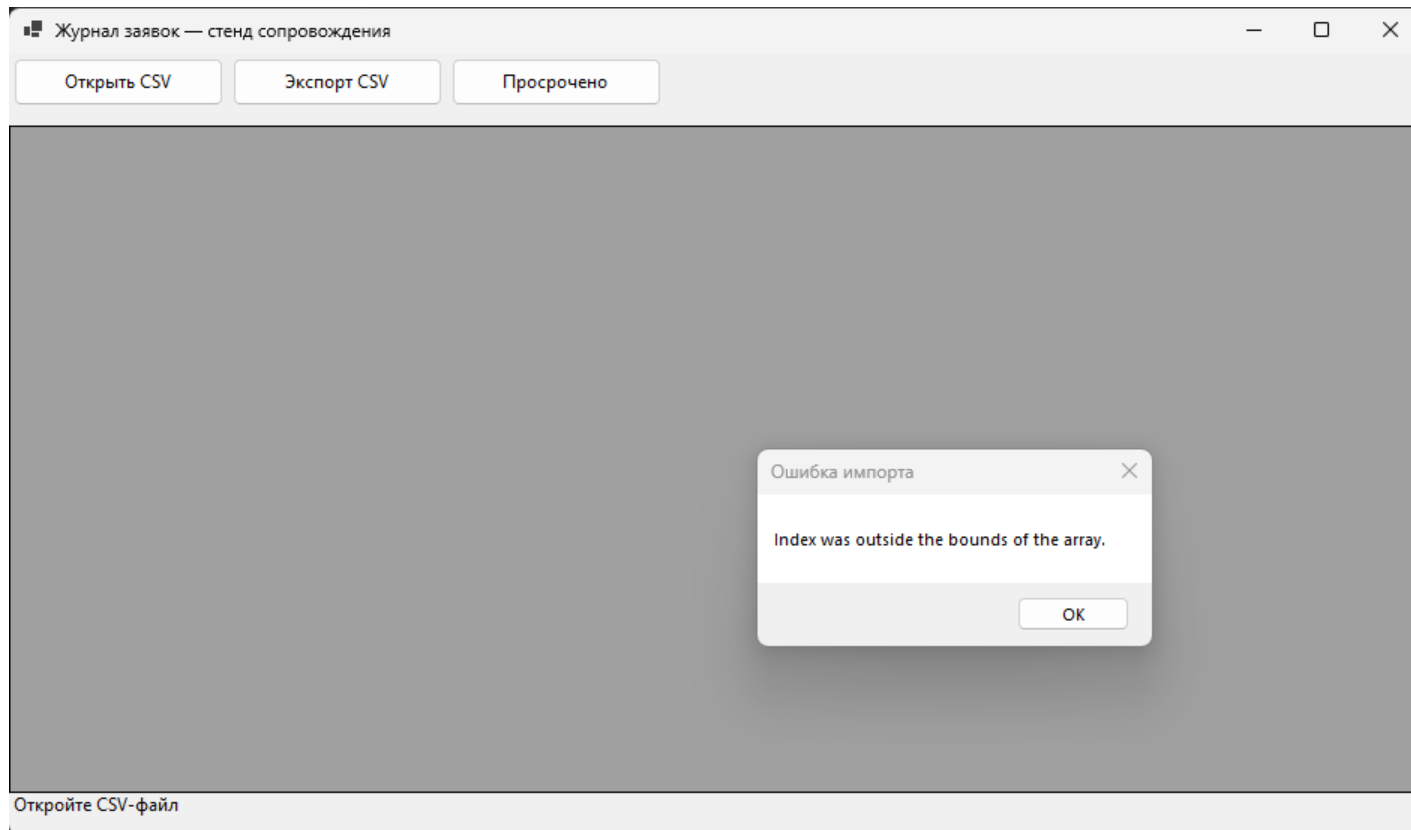
Не перепутайте причины

В заготовке есть намеренные дефекты. Её назначение — дать воспроизводимый симптом для сопровождения. Работайте на отдельной копии; проверенный стартовый проект с кнопкой «Тестовые данные» сохраните для регрессии. CSV-версия использует другие кнопки, поэтому сохранность исходного поведения проверяется сравнением двух копий, а не поиском старой кнопки в новой форме. В конце методички приведён исправленный пример, но сначала воспроизведите проблемы.

Путь к данным

Скачайте demo.csv по прямой ссылке. Он не находится внутри исходного ZIP: файл выдаётся отдельно, чтобы было видно момент подключения внешних данных. Откройте его как текст, проверьте заголовок и неполную строку 4. Файл demo-valid.csv помогает отделить дефект экспорта от дефекта импорта.

Реальная ошибка первой интеграции



Черновая интеграция CSV, сборка .NET 10 на Windows: неполная строка 4 прерывает импорт, окно остаётся пустым.

Контракт данных перед чтением кода

Материалы для работы

[Скачать демонстрационный CSV с неполной строкой \(demo.csv\)](#)

Используйте для воспроизведения неполной строки и исправленного импорта.

[Скачать демонстрационный CSV без дефектной строки \(demo-valid.csv\)](#)

Используйте для отдельной проверки экспорта и повторного импорта.

[Скачать заготовку подключения CSV \(ZIP\)](#)

Откройте после первой проверки, чтобы изучить дефекты импорта и экспорта.

Пять полей и их смысл

Id — номер заявки; Item — объект, например «Проектор»; Owner — ответственный; Due — контрольная дата; Status — «Открыта» или «Завершена». Первая строка Id;Item;Owner;Due;Status обязательна. Сохраните точку с запятой как разделитель. Обычная запятая и локальный формат даты здесь не являются частью договора.

Проверка файла глазами

В demo.csv строки 2 и 3 имеют по пять полей, а строка 4 заканчивается после Owner. Перед запуском напишите ожидаемое: две корректные записи должны пережить ошибку соседней строки, предупреждение должно назвать строку 4. Сравните с текущей заготовкой: исключение появляется раньше обновления таблицы.

Где искать первопричину

В заготовке откройте Program.cs и найдите CsvStore.Read. Посмотрите, как создаётся массив cells и где читается cells[4]. Доступ к несуществующей ячейке объясняет ошибку. Следующий раздел показывает безопасный порядок: заголовок файла проверяется один раз, каждая запись валидируется до преобразования.

Модель результата чтения

Где выполнять действие

Откройте Program.cs исправленного демонстрационного проекта из ServiceDesk-fixed-example.zip. Первые строки показывают необходимые пространства имён и модель ReadResult. В своей рабочей копии добавьте using System.Globalization и using System.Text в начале файла; запись ReadResult разместите рядом с существующим Ticket до CsvStore. Не создавайте второй Ticket и не переносите namespace ServiceDeskFixed в проект ServiceDesk: сохраните собственное пространство имён.

Program-fixed.cs: строки 1–9

```
using System.Globalization;
using System.Text;
using System.Windows.Forms;

namespace ServiceDeskFixed;

internal record Ticket(int Id, string Item, string Owner, DateTime Due, string Status);
internal record ReadResult(List<Ticket> Tickets, List<string> Warnings);
```

Почему это работает и как проверить

ReadResult передаёт два независимых результата: корректные Tickets и Warnings. Это позволяет форме показать записи, даже когда одна строка ошибочна. После добавления типов соберите проект: ошибки имён или using должны быть устранены до изменения алгоритма.

Устойчивое чтение файла

Где выполнять действие

В рабочем Program.cs создайте CsvStore после записей Ticket и ReadResult. Внутри него добавьте Header и начало метода Read. Заголовок проверяется до цикла; неверный заголовок делает весь файл непригодным.

Program-fixed.cs: строки 10–20

```
internal static class CsvStore
{
    private const string Header = "Id;Item;Owner;Due;Status";

    public static ReadResult Read(string path)
    {
        var tickets = new List<Ticket>();
        var warnings = new List<string>();
        var lines = File.ReadAllLines(path, Encoding.UTF8);
        if (lines.Length == 0 || lines[0].TrimStart('\uFEFF') != Header)
            throw new InvalidDataException($"Ожидается заголовок {Header}");
    }
}
```

Почему это работает и как проверить

Здесь создаются два списка: принятые записи и предупреждения. Фрагмент обрывается перед циклом, поэтому завершите метод на следующей странице и только затем собирайте проект. После завершения на файле только с заголовком метод вернёт два пустых списка, а при неверном заголовке выдаст общую ошибку.

Валидация одной строки без потери остальных

Где выполнять действие

Продолжите метод Read после проверки заголовка. В цикле сначала Split, затем проверка Length и только после неё обращения к полям. Для Id и даты используйте TryParse, для пустых полей и статуса — явные условия.

Program-fixed.cs: строки 21–37

```
for (int index = 1; index < lines.Length; index++)
{
    if (string.IsNullOrWhiteSpace(lines[index])) continue;
    var cells = lines[index].Split(';');
    string? reason = null;
    if (cells.Length != 5) reason = "ожидается пять полей";
    else if (!int.TryParse(cells[0], out _)) reason = "неверный Id";
    else if (string.IsNullOrWhiteSpace(cells[1]) || string.IsNullOrWhiteSpace(cells[2])) reason = "пустое обязательное поле";
    else if (!DateTime.TryParseExact(cells[3], "dd.MM.yyyy", CultureInfo.InvariantCulture, DateTimeStyles.None, out _)) reason = "неверная да
та";
    else if (cells[4] != "Открыта" && cells[4] != "Завершена") reason = "неверный статус";
    if (reason != null) { warnings.Add($"Строка {index + 1}: {reason}"); continue; }
    tickets.Add(new Ticket(int.Parse(cells[0], CultureInfo.InvariantCulture), cells[1], cells[2],
        DateTime.ParseExact(cells[3], "dd.MM.yyyy", CultureInfo.InvariantCulture), cells[4]));
}
return new ReadResult(tickets, warnings);
}
```

Почему это работает и как проверить

Предупреждение содержит index + 1, потому что индекс массива начинается с нуля, а строка файла — с единицы. continue пропускает только текущую запись. На demo.csv ожидаются 101 и 102 плюс предупреждение о строке 4. Для неверной даты создайте копию файла и проверьте, что оставшаяся запись видна.

Экспорт в совместимом формате

Где выполнять действие

В том же CsvStore после Read добавьте Write. Откройте demo-valid.csv и сравните точный заголовок. Метод должен сначала записать Header, затем сформировать пять полей каждой заявки. До записи проверьте текстовые поля на разделитель и перевод строки.

Program-fixed.cs: строки 38–49

```
public static void Write(string path, IEnumerable<Ticket> tickets)
{
    var lines = new List<string> { Header };
    foreach (var t in tickets)
    {
        if (new[] { t.Item, t.Owner, t.Status }.Any(value => value.Contains(';') || value.Contains('\n') || value.Contains('\r')))
            throw new InvalidDataException("В поле нельзя использовать разделитель или перевод строки");
        lines.Add($"{t.Id};{t.Item};{t.Owner};{t.Due.ToString("dd.MM.yyyy", CultureInfo.InvariantCulture)};{t.Status}");
    }
    File.WriteAllLines(path, lines, Encoding.UTF8);
}
```

Почему это работает и как проверить

Формат dd.MM.yyyy с InvariantCulture не зависит от настроек компьютера. После сборки экспортируйте два корректных значения и снова импортируйте результат; сравните все пять полей. Если название объекта содержит точку с запятой, получите понятную ошибку вместо молчаливо повреждённого CSV.

Подсчёт просроченных

Где выполнять действие

После Write в CsvStore добавьте CountOverdue. Сравните выражение с заготовкой: одной проверки Due недостаточно. Условие Status != «Завершена» оставляет в выборке только незавершённые записи; Due.Date < today.Date исключает сегодняшнюю дату.

Program-fixed.cs: строки 50–53

```
public static int CountOverdue(IEnumerable<Ticket> tickets, DateTime today) =>
    tickets.Count(t => t.Status != "Завершена" && t.Due.Date < today.Date);
}
```

Почему это работает и как проверить

Проверьте три независимых случая: открытая запись с прошедшей датой учитывается; завершённая с прошедшей датой и открытая с сегодняшней — нет. Это проверка правила оператора, а не синтаксиса CSV.

СВЯЗЬ МЕТОДОВ С ОКНОМ

Где выполнять действие

В исправленном Program.cs найдите MainForm. В рабочей копии оставьте таблицу, строку состояния и список tickets от стартового приложения. Ниже полей начинается конструктор формы.

Program-fixed.cs: строки 54–64

```
internal sealed class MainForm : Form
{
    private readonly DataGridView grid = new() { Dock = DockStyle.Fill, ReadOnly = true, AutoGenerateColumns = true };
    private readonly Label status = new() { Dock = DockStyle.Bottom, Height = 28, Text = "Откройте CSV-файл" };
    private List<Ticket> tickets = new();

    public MainForm(string? demoPath = null)
    {
        Text = "Журнал заявок – версия 1.1";
        Width = 960;
        Height = 560;
    }
}
```

Почему это работает и как проверить

Сохранённая таблица показывает, что сопровождение расширяет проверенный проект. Проверьте имена grid, status и tickets до подключения кнопок.

Кнопки обмена и счётчика

Где выполнять действие

В конструкторе MainForm добавьте на панель кнопки «Открыть CSV», «Экспорт CSV», «Просрочено» и подпишите Click соответствующими методами. Список tickets остаётся источником данных для таблицы и расчёта.

Program-fixed.cs: строки 65–78

```
var bar = new FlowLayoutPanel { Dock = DockStyle.Top, Height = 48 };
var open = new Button { Text = "Открыть CSV", Width = 140, Height = 32 };
var save = new Button { Text = "Экспорт CSV", Width = 140, Height = 32 };
var count = new Button { Text = "Просрочено", Width = 140, Height = 32 };
open.Click += (_, _) => OpenCsv();
save.Click += (_, _) => SaveCsv();
count.Click += (_, _) => status.Text = $"Просрочено: {CsvStore.CountOverdue(tickets, DateTime.Today)}";
bar.Controls.AddRange(new Control[] { open, save, count });
Controls.Add(grid);
Controls.Add(status);
Controls.Add(bar);
if (demoPath != null) Shown += (_, _) => LoadCsv(demoPath);
}
```

Почему это работает и как проверить

После завершения обработчиков проверьте, что «Просрочено» вызывает CsvStore.CountOverdue для текущего списка, а не для демонстрационных данных.

Диалог открытия и сообщения о строках

Где выполнять действие

После конструктора MainForm добавьте OpenCsv. OpenFileDialog выбирает путь; при отмене диалога метод ничего не меняет. Передайте выбранный путь в LoadCsv.

Program-fixed.cs: строки 79–85

```
private void OpenCsv()
{
    using var dialog = new OpenFileDialog { Filter = "CSV (*.csv)|*.csv" };
    if (dialog.ShowDialog() != DialogResult.OK) return;
    LoadCsv(dialog.FileName);
}
```

Почему это работает и как проверить

При нажатии «Открыть CSV» должен появиться фильтр файлов CSV. Отмена не должна очищать текущую таблицу.

Показ записей и предупреждений

Где выполнять действие

После OpenCsv добавьте LoadCsv. Он вызывает CsvStore.Read: сначала сохраняет Tickets в поле формы и обновляет DataGridView, затем показывает Warnings. Общую ошибку отсутствующего файла ловит как исключение импорта.

Program-fixed.cs: строки 86–98

```
private void LoadCsv(string path)
{
    try
    {
        var result = CsvStore.Read(path);
        tickets = result.Tickets;
        grid.DataSource = tickets.ToList();
        status.Text = $"Загружено: {tickets.Count}; пропущено: {result.Warnings.Count}";
        if (result.Warnings.Count > 0) MessageBox.Show(string.Join(Environment.NewLine, result.Warnings), "Предупреждения импорта");
    }
    catch (Exception ex) { MessageBox.Show(ex.Message, "Ошибка импорта"); }
}
```

Почему это работает и как проверить

При demo.csv форма показывает две записи, статус — «Загружено: 2; пропущено: 1», предупреждение — строку 4. После закрытия окна таблица остаётся заполненной.

Проверка результата импорта в реальном окне

Материалы для работы

[Скачать демонстрационный CSV с неполной строкой \(demo.csv\)](#)

Используйте для воспроизведения неполной строки и исправленного импорта.

[Скачать демонстрационный CSV без дефектной строки \(demo-valid.csv\)](#)

Используйте для отдельной проверки экспорта и повторного импорта.

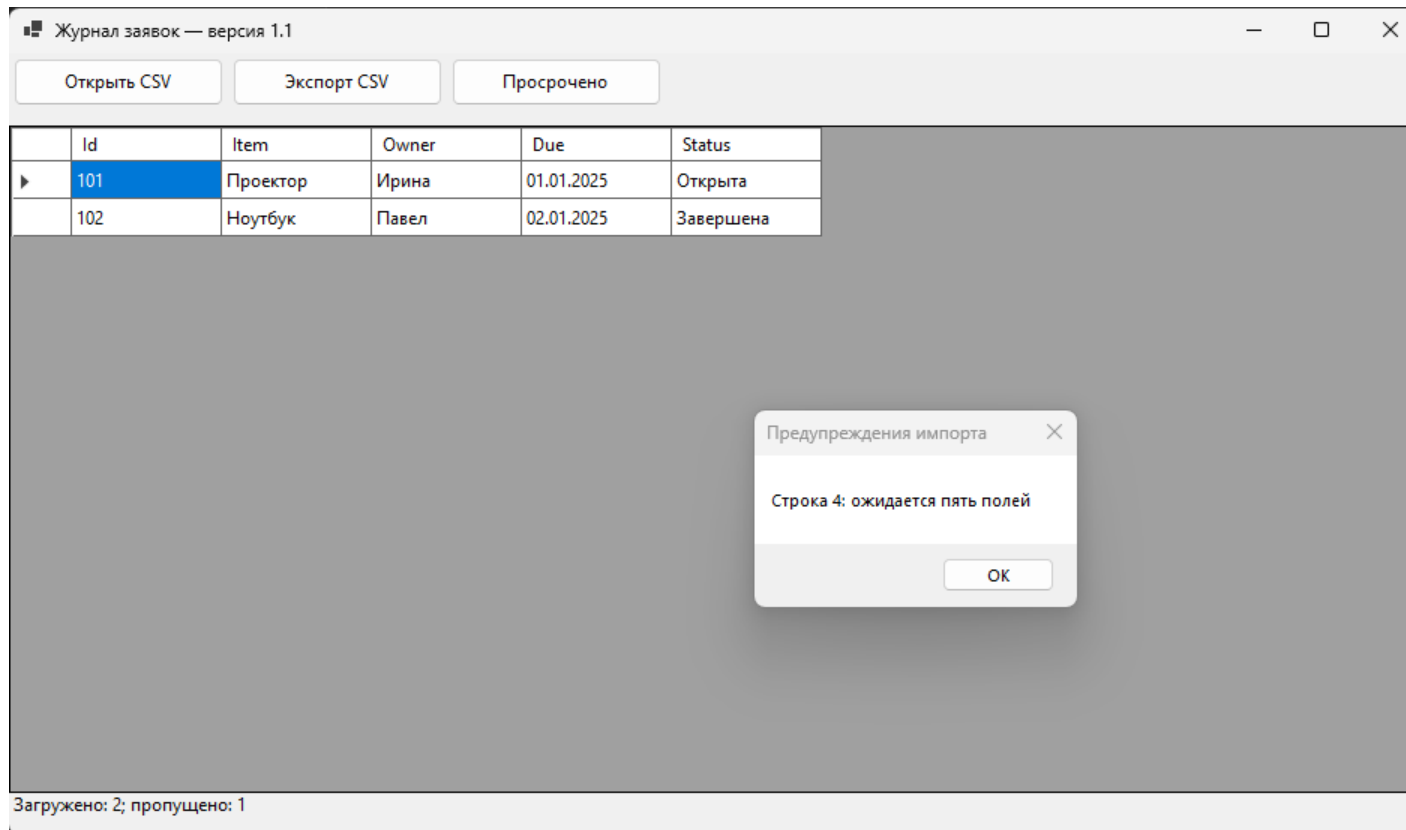
Повторите без подмены данных

В запущенной исправленной рабочей копии выберите именно demo.csv. Сравните текст предупреждения, число строк и значения Item/Owner с исходным файлом. Затем откройте demo-valid.csv: предупреждений нет, обе записи остаются видимыми. Если таблица пуста, проверьте, присвоен ли grid.DataSource после вызова Read.

Почему предупреждение не блокирует оператора

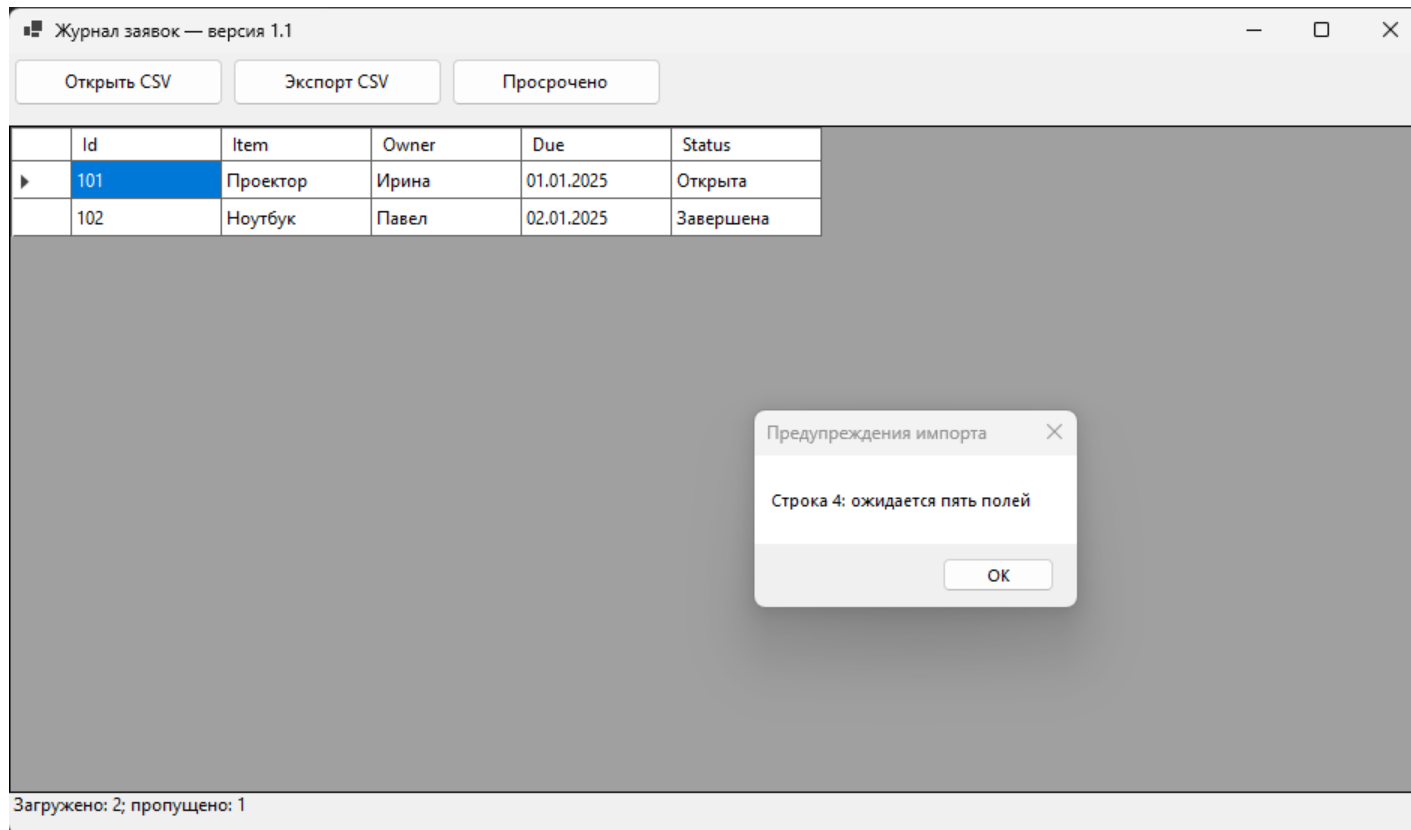
Ошибка строки 4 сообщает о проблеме данных, но не делает заявки 101 и 102 бесполезными. Оператор должен продолжить работу с ними, сохранив информацию о пропуске для исправления CSV. Проверка видимости строк после закрытия окна подтверждает это поведение.

Результат исправленного импорта



Исправленная демонстрационная сборка: две корректные записи сохранены, строка 4 пропущена с предупреждением.

Две доступные записи



После закрытия предупреждения таблица содержит проектор и ноутбук; нижняя строка показывает 2 загруженные и 1 пропущенную запись.

Сохранение и запуск проверок

Где выполнять действие

После LoadCsv добавьте SaveCsv с SaveFileDialog; сохранение выполняйте в новый файл. Запись передаётся CsvStore.Write. Отмена диалога не должна менять данные.

Program-fixed.cs: строки 99–106

```
private void SaveCsv()
{
    using var dialog = new SaveFileDialog { Filter = "CSV (*.csv)|*.csv", FileName = "export.csv" };
    if (dialog.ShowDialog() != DialogResult.OK) return;
    try { CsvStore.Write(dialog.FileName, tickets); status.Text = "Файл сохранён"; }
    catch (Exception ex) { MessageBox.Show(ex.Message, "Ошибка экспорта"); }
}
```

Почему это работает и как проверить

В GUI нажмите «Экспорт CSV», затем «Открыть CSV» и сравните строки. Исходный CSV при этом не перезаписывается.

Самопроверка обмена

Где выполнять действие

В конце MainForm добавьте точку входа Main. Режим --self-test принимает путь к CSV, выполняет импорт, экспорт во временный файл и повторное чтение. Этот код остаётся в проверенном демонстрационном примере.

Program-fixed.cs: строки 107–122

```
[STAThread]
private static void Main(string[] args)
{
    if (args.Length == 2 && args[0] == "--self-test")
    {
        var result = CsvStore.Read(args[1]);
        var target = Path.GetTempFileName();
        try
        {
            CsvStore.Write(target, result.Tickets);
            var roundtrip = CsvStore.Read(target);
            if (roundtrip.Warnings.Count != 0 || !result.Tickets.SequenceEqual(roundtrip.Tickets))
                throw new InvalidOperationException("Повторный импорт не совпал");
            Console.WriteLine($"tickets={result.Tickets.Count}; warnings={result.Warnings.Count}; overdue={CsvStore.CountOverdue(result.Tickets,
DateTime.Today)}");
            foreach (var warning in result.Warnings) Console.WriteLine(warning);
        }
    }
}
```

Почему это работает и как проверить

Выполните `dotnet run --project ServiceDesk-fixed.csproj -- --self-test demo.csv`: ожидаются `tickets=2` и `warnings=1`. Если обратное чтение отличается, программа выдаёт ошибку вместо сообщения об успехе.

Удаление временного файла и запуск GUI

Где выполнять действие

Завершите Main: блок `finally` удаляет временный экспорт, затем обычная ветка инициализирует Windows Forms и запускает `MainForm`. Разместите эти строки после тела самопроверки, не создавая вторую точку входа.

Program-fixed.cs: строки 123–129

```
        finally { File.Delete(target); }  
        return;  
    }  
    ApplicationConfiguration.Initialize();  
    Application.Run(new MainForm(args.Length == 2 && args[0] == "--demo" ? args[1] : null));  
}
```

Почему это работает и как проверить

После проверки временный файл удалён, исходный CSV не затронут. Запуск F5 по-прежнему открывает форму; режим `--demo` ПУТЬ_K_CSV показывает данные в окне.



Регрессия и граничные наборы

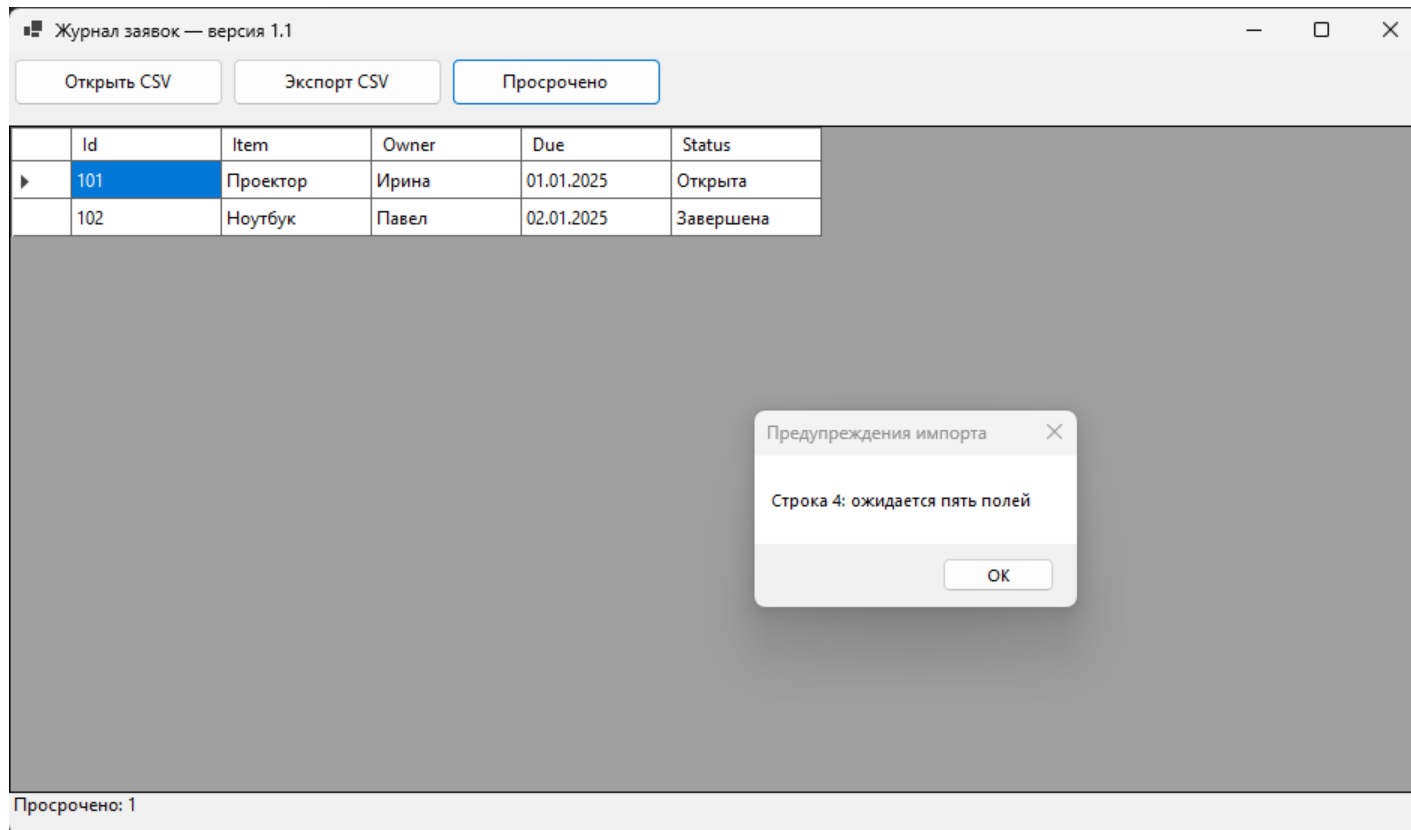
Матрица до запуска

Для каждого случая запишите вход, ожидаемое число записей, предупреждение и состояние счётчика. Используйте копии CSV: неполная строка, неверный Id, неверная дата, пустой файл с заголовком, завершённая заявка с прошлой датой, открытая с датой сегодня. Не изменяйте исходный demo.csv: он остаётся повторяемой контрольной точкой.

Сравнение до и после

Повторите исходные действия с кнопками после каждой правки. Проверка импорта не подтверждает экспорт; поэтому обязательно откройте сохранённый файл тем же приложением. Проверка даты не подтверждает сохранность полей; сравните Id, Item, Owner, Due и Status построчно. Если одно условие не выполнено, вернитесь к соответствующему методу и выясните причину.

Проверка просрочки



В исправленной версии завершенная заявка не входит в счётчик: показано «Просрочено: 1».

Переход к индивидуальной предметной области

Материалы для работы

[Скачать исправленный демонстрационный проект \(ZIP\)](#)

Сверяйтесь после самостоятельной диагностики; это решение демонстрационной области.

Что менять

На странице своего варианта скачайте его CSV. В копии стартового проекта замените демонстрационные значения Item и Owner на объекты и ответственных области, проверьте встроенные записи. После этой контрольной точки добавьте CSV-обмен. Заголовок файла и пять полей модели сохраняются: это позволяет повторно использовать проверенные методы Read и Write.

Что проверить отдельно

В индивидуальном CSV две корректные записи и одна неполная. Проверьте D1–D3 на этих данных, затем особое правило области. Оно связано с её действием и объектом, поэтому одного снимка демонстрационного «Проектора» недостаточно. В отчёте сопоставьте правило, тестовый вход и фактический результат.

Как использовать исправленный пример

ServiceDesk-fixed-example.zip показывает рабочий результат только для учебного центра. Сравнивайте устройство методов и порядок проверок после собственной диагностики. Не заменяйте свой вариант этим архивом: названия объектов, ответственных и индивидуальная проверка должны соответствовать вашему CSV.



Обновление, откат и инструкция оператору

Проверка на чистой копии

Сохраните исходный CSV отдельно, затем разверните исправленную сборку в новую папку. Запустите приложение, импортируйте копию данных, проверьте экспорт и счётчик. Если новый пакет не запускается или результаты расходятся, верните предыдущую программу и восстановите исходный CSV. Эта репетиция нужна до передачи пользователю.

Текст инструкции

Для оператора опишите последовательность «Открыть CSV → прочитать предупреждение → проверить строки → Экспорт CSV → найти сохранённый файл». Добавьте правило подсчёта просрочки и способ сообщить о повреждённой строке. Названия кнопок должны совпадать с окном; технические имена классов оператору не нужны.

Завершение анализа

Свяжите D1, D2 и D3 с конкретными строками исходника, исправлением и повторной проверкой. Укажите, что осталось ограничением: этот учебный формат не поддерживает точку с запятой внутри текстового поля. При сдаче проекта приложите ссылку на его репозиторий; результаты испытаний описывайте только по фактическим запускам.